

BRAC University

Department of Computer Science and Engineering

CSE111: Programming Language II

Examination: *Theory Assignment #1*

Semester: *Summer 2022*

Date: ____ / ____ / 2022

Deadline: 8 August, 2022

ID: _____	Name: (Please write in CAPITAL LETTERS)	Section: 42
--------------	--	-----------------------

1. Recently, your friend found an entry-level Software Engineering job position at Lazofth Inc. The company mainly works with Spring - an open-source Java framework - and requires the applicant must know the basics of Java at least.

Your friend is an expert in Python but has never written any program in Java. So, he is learning from a Java enthusiast who also knows Python well enough. The first, assignment he received is to reverse engineer from a driver-code and output of a Python code that follows Java programming style.

The task is extremely easy for him, however, your midterm exams are over but his exams aren't so, he will try again later. Thus, he is seeking help from you to complete the assignment just to meet the deadline. For your convenience, he has provided additional information such as **Class Diagram**.

Java	System	Scanner	PrintStream
system: System	out:PrintStream		
<code>__init__()</code> <code>_while()</code> <code>scanner()</code> <code>toString()</code>	<code>__init__()</code>	<code>nextInt()</code> <code>nextFloat()</code> <code>nextLine()</code>	<code>print()</code> <code>println()</code> <code>printf()</code>

Driver Code:

Write your codes here

```
java = Java()
sc = java.scanner()
your_name = sc.nextLine() # <YOUR_NAME>
java.system.out.println("Hello " + your_name)
your_nationality = sc.nextLine("Nationality: ") # <BANGLADESHI>
java.system.out.println("You are " + your_nationality)
first_number = sc.nextInt() # 3
second_number = sc.nextInt() # 8
fstr = java.toString(first_number)
sstr = java.toString(second_number)
java.system.out.print("Summation of " + fstr + " and " + sstr + ": ")
java.system.out.println(first_number + second_number)
```

```

java._while(start=first_number, # You may use exec() function.
             operator="<",
             end=second_number,
             step=1,
             data="Welcome Java in Python")
height = 7.187342
java.system.out.printf("%.2f", height)
java.system.out.println("Hello world")
java.system.out.println("One more time!")
java.system.out.printf("%.2f\n", height)
java.system.out.println("Hello world")
java.system.out.println("Done!")
java._while()

```

Output:

```

Please enter a string value: Zulfikar
Hello Zulfikar
Nationality: Bangladeshi
You are Bangladeshi
Please enter an integer value: 3
Please enter an integer value: 8
Summation of 3 and 8: 11
Welcome Java in Python
Welcome Java in Python
Welcome Java in Python
Welcome Java in Python
Welcome Java in Python
7.19Hello world
One more time!
7.19
Hello world
Done!
Java API 8
Java API 8
Java API 8
Java API 8
Java API 8

```

2. All right! Your exams are finally finished and you helped your friend with his assignment. Now, you need to get back to your studies. And since you find Question 1 of Quiz #2 pretty interesting and want to create a GUI application similar to that but couldn't answer it accurately during the exam, you are ready to try it again!

Therefore, you asked your teacher to give you the question with the increased features, if possible. And you get the following from your teacher:

Driver Code:

```

# Write your class design code here

cse111_quiz = ExamTracker("CSE111", "Quiz")
cse111_quiz.add_topics("String Manipulations", module="1")

```

```

cse111_quiz.add_topics("List and Tuple", module="1")
cse111_quiz.add_topics("Dictionary", exam_date="15/06/2022",
                        module="1")
cse111_quiz.add_topics("Functions", module="1")
cse111_quiz.update_date("20/06/2022", module="1")
cse111_quiz.add_topics("OOP Basics", exam_date="04/07/2022",
                        module="2")
cse111_quiz.add_topics("Instance Variables and Functions",
                        "Method Overloading", module="2")
cse111_mid = ExamTracker("CSE111", "Midterm", "19/07/2022")
print(cse111_mid.get_details())
cse111_mid.add_topics(*cse111_quiz.get_topics())
cse111_mid.print_details()
cse111_quiz.add_topics("Operator Overloading",
                        "Encapsulation",
                        "Class Variable",
                        "Class & Static Method",
                        module="3")
cse111_quiz.add_topics("Inheritance", "Method Overriding",
                        "Variable Overriding", "Abstract",
                        exam_date="22/08/2022", module="4")
cse111_quiz.update_date("10/08/2022", module="3")
print(cse111_quiz.get_details())
cse111_final = ExamTracker("CSE111", "Final", "02/09/2022")
cse111_final.add_topics(*cse111_quiz.get_topics())
cse111_final.print_details()
print(cse111_quiz.get_topics())
print("===== GOOD LUCK =====")
print(cse111_quiz.get_topics("3"))

```

Output:

```

New ExamTracker for CSE111 - Quiz has been created...
CSE111 exam date for Quiz - 1 has been changed from 15/06/2022 to 20/06/2022
New ExamTracker for CSE111 - Midterm has been created...
=====

```

```

Course: CSE111
Exam Title: Midterm
Modules:
    No modules have been created yet.

```

```

=====
Course: CSE111
Exam Title: Midterm
Modules:
    Midterm - default
    Exam Date: 19/07/2022
        - String Manipulations
        - List and Tuple
        - Dictionary
        - Functions
        - OOP Basics
        - Instance Variables and Functions
        - Method Overloading

```

```

CSE111 exam date for Quiz - 3 has been changed from Not Defined to 10/08/2022

```

=====

Course: CSE111

Exam Title: Quiz

Modules:

Quiz - 1

Exam Date: 20/06/2022

- String Manipulations
- List and Tuple
- Dictionary
- Functions

Quiz - 2

Exam Date: 04/07/2022

- OOP Basics
- Instance Variables and Functions
- Method Overloading

Quiz - 3

Exam Date: 10/08/2022

- Operator Overloading
- Encapsulation
- Class Variable
- Class & Static Method

Quiz - 4

Exam Date: 22/08/2022

- Inheritance
- Method Overriding
- Variable Overriding
- Abstract

New ExamTracker for CSE111 - Final has been created...

=====

Course: CSE111

Exam Title: Final

Modules:

Final - default

Exam Date: 02/09/2022

- String Manipulations
- List and Tuple
- Dictionary
- Functions
- OOP Basics
- Instance Variables and Functions
- Method Overloading
- Operator Overloading
- Encapsulation
- Class Variable
- Class & Static Method
- Inheritance
- Method Overriding
- Variable Overriding
- Abstract

['String Manipulations', 'List and Tuple', 'Dictionary', 'Functions', 'OOP Basics', 'Instance Variables and Functions', 'Method Overloading', 'Operator Overloading', 'Encapsulation', 'Class Variable', 'Class & Static Method', 'Inheritance', 'Method Overriding', 'Variable

```
Overriding', 'Abstract']
===== GOOD LUCK =====
['Operator Overloading', 'Encapsulation', 'Class Variable', 'Class &
Static Method']
```

3. There are limited resources for practicing Object Oriented Programming (OOP), unlike Procedural Programming. Therefore, you have decided to look around and create a question for yourself. Why not start with BRAC University?

After visiting: <https://www.bracu.ac.bd/academics/departments>, you thought about working with it. You have already written the driver code and the expected output. Now, to make sure your reverse engineering knowledge is as good as you think it is, you will not change anything (even not a character) from the driver code or the output and design a multiclass OOP to complete the task. For your convenience, you have already drawn the class diagram to specify the instance variables and methods:

Course
code: str title: str credit: float department: Department
<code>__init__(): None</code> <code>update(): None</code> <code>to_string(): str</code>

Department
abbreviation: str title: str courses: [Course]
<code>__init__(): None</code> <code>get_course_by_code(): Course</code> <code>register_courses(): None</code> <code>remove_courses(): None</code> <code>to_string(): str</code>

Driver code:

```
# Write your code here

cse = Department("CSE", "Computer Science and Engineering")
enh = Department("ENH", "English and Humanities")
mns = Department("MNS", "Mathematics and Natural Sciences")
cse110 = Course("CSE110", "Programming Language I", 3.0, cse)
cse111 = Course("CSE111", "Programming Language II", 3.0, cse)
eng091 = Course("ENG091", "Foundation Course", 0, enh)
eng101 = Course("ENG101", "English Fundamentals", 3.0, enh)
mat092 = Course("MAT092", "Remedial Course", 0.0, mns)
mat216 = Course("MAT216", "Linear Algebra and Fourier Analysis", 3, mns)
phy111 = Course("PHY111", "Principles of Physics I", 3, mns)
cse221 = Course("CsE112", "Algorithms", 0, mns)

print(cse.to_string())
print(enh.to_string())
print(mns.to_string())

cse221.update(code="CSE221", credit=3, department=cse)

print("\n", "=" * 25, sep="\n")
```

```

print("||" + " " * 5 + "Summer 2022" + " " * 5 + "||")
print("=" * 25, "\n", sep="\n")
print(cse.to_string())
print(mns.to_string())
print(cse221 == cse.get_course_by_code("CSE221"))
print(cse.get_course_by_code("CSE110").to_string())

```

Output:

```

Department of Computer Science and Engineering [CSE]
Courses: 2
Credits: 6.0
  - CSE110 [3.0] | Programming Language I
  - CSE111 [3.0] | Programming Language II
=====
Department of English and Humanities [ENH]
Courses: 2
Credits: 3.0
  - ENG091 [0.0] | Foundation Course
  - ENG101 [3.0] | English Fundamentals
=====
Department of Mathematics and Natural Sciences [MNS]
Courses: 4
Credits: 6.0
  - MAT092 [0.0] | Remedial Course
  - MAT216 [3.0] | Linear Algebra and Fourier Analysis
  - PHY111 [3.0] | Principles of Physics I
  - CsE112 [0.0] | Algorithms
=====

=====
||      Summer 2022      ||
=====

Department of Computer Science and Engineering [CSE]
Courses: 3
Credits: 9.0
  - CSE110 [3.0] | Programming Language I
  - CSE111 [3.0] | Programming Language II
  - CSE221 [3.0] | Algorithms
=====
Department of Mathematics and Natural Sciences [MNS]
Courses: 3
Credits: 6.0
  - MAT092 [0.0] | Remedial Course
  - MAT216 [3.0] | Linear Algebra and Fourier Analysis
  - PHY111 [3.0] | Principles of Physics I
=====
True
CSE110 [3.0] | Programming Language I

```

4. Trace the following code:

```
class Task4:
    def __init__(self):
        self.a, self.b, self.c = ["a"], ["b"], ["c"]
        self.d = ([self.a, self.b, self.c])
        self.bin = [0, 1, 1]; self.i = 2

    def method_one(self):
        self.bin[-1] += 1
        for i in range(1, len(self.bin) + 1):
            self.bin[-i] += self.bin[1 - i] // 2
        for i in range(len(self.bin)): self.bin[i] %= 2
        self.d[self.i].append(self.method_two(self.d))
        a, b, c = self.d; print(*self.bin); print(a, b, c)

    def method_two(self, d):
        self.i = self.next(self.i)
        i, j = self.next(self.i), self.next(self.i + 3)
        d[i], d[j] = d[j], d[i]
        return int("".join([str(i) for i in self.bin]), 2)

    def next(self, i):
        return (i + 1) % len(self.d)
```

What is the output of the following code sequence?

```
t4 = Task4()
t4.method_one()
t4.method_one()
t4.method_one()
t4.method_one()
```

5. Trace the following code:

```
class MsgClass:
    def __init__(me, self):
        self.x, self.y = 1, 2

class Task5:
    def __init__(summer):
        summer.x, summer.y = 3, 4

    def method_one(me, other):
        myMsg = MsgClass(me)
        sum = me.x + other.y
        print(other.x, me.y, sum)
        sum = other.method_two() + me.method_two(other)
        print(me.x, other.y, sum)

    def method_two(self, other=None):
        self.x += 10; self.y += 20
        if other: self = other
        self.x += 2; self.y += 3
        return self.x + self.y
```

What is the output of the following code sequence?

```
t1 = Task5()
t1.method_one(t1)
t2 = Task5()
t2.method_one(t1)
t3 = Task5()
t3.method_one(t1)
```